

✓ Relatório Técnico

1. Visão geral

O sistema de monitoramento e diagnóstico em tempo real desenvolvido em Python, projetado para simular o controle operacional de uma estação espacial ou base remota. O objetivo central é interpretar dados de telemetria provenientes de sensores e módulos de subsistema, classificar automaticamente o estado da missão e emitir alertas e recomendações de ação imediata.

2. Arquitetura

O repositório está organizado em três camadas:

`data/`

- `dados.csv` - conjunto de telemetria;
- `simuladorDados.py` — gerador sintético de registros.

`src/`

- `sistema.py` - núcleo do sistema: entrada de dados, processamento, diagnóstico e saída.

`docs/`

- `relatorio.pdf` - Relatório técnico;
- `link_video.txt` - Link do vídeo no youtube.

3. Estruturas de Dados

- **deque** — Fila de Alertas do Sistema

Uso: Variável global `fila_alertas_sistema`.

A deque da biblioteca `collections` oferece inserções e remoções $O(1)$ em ambas as extremidades. No sistema é utilizada exclusivamente com semântica FIFO: alertas são inseridos pela direita (`append()`) e consumidos pela esquerda (`popleft()`), garantindo exibição na mesma ordem de prioridade em que foram enfileirados.

- **Lista como Pilha** — Eventos Críticos

Uso: Variável global `pilha_eventos_criticos`.

Python nativo implementa pilha LIFO com listas comuns usando `append()` (push) e acesso pelo índice `[-1]` (peek). Cada falha de módulo detectada é empilhada com registro de horário. O último evento crítico é exibido na interface, favorecendo a investigação pós-incidente sem necessidade de varredura linear.

- **Matriz (Lista de Listas)** — Histórico de Telemetria

Uso: Variável global `matriz_historico`.

O CSV é materializado linha a linha como uma lista de listas, onde `matriz_historico[0]` contém o cabeçalho e as linhas seguintes os valores convertidos para seus tipos nativos (int, float ou str). Essa representação tabular permite acesso $O(1)$ por índice de linha e facilita futuras extensões como análise de séries temporais completas ou exportação.

- **Dicionários** — Hierarquia de Dados Organizados

Uso: Retorno de `organizarDados()` e variável `situacao` em `main()`.

Os dados brutos são reestruturados em um dicionário hierárquico com quatro sub-domínios:

1. `hierarquia.energia` — dados de geração e reserva;
2. `hierarquia.habitat` — temperatura interna/externa e radiação;
3. `módulos` — status binário (0/1) de seis subsistemas;
4. `sensores` — valores escalares para lógica de diagnóstico e previsão.

O acesso por chave nomeada elimina indexação posicional frágil e torna o código auto-documentado.

Estrutura	Tipo Python	Padrão de Acesso	Complexidade
<code>fila_alertas_sistema</code>	deque	FIFO (enfileirar/desenfileirar)	$O(1)$
<code>pilha_eventos_criticos</code>	list (pilha)	LIFO (push/peek)	$O(1)$
<code>matriz_historico</code>	list of lists	Acesso por índice de linha	$O(1)$
<code>organizados / situacao</code>	dict aninhado	Acesso por chave nomeada	$O(1)$ médio

4. Lógica de Diagnóstico e Classificação

O coração das decisões do sistema está na função

`classificarMissao(situacao)`, que aplica lógica booleana de três níveis para determinar o estado global da missão.

- **Expressão Booleana para Estado CRÍTICO**

```

1
2 falhas_criticas = sum(1 for v in [modulos['suporteVida'], modulos['energia'], modulos['habitacao']] if v == 0)
3 is_critico = (falhas_criticas >= 1) or (sensores['energiaRestante'] < 20) or (sensores['radiacao'] > 13.0)

```

Qualquer uma das três condições é suficiente para elevar o status a CRÍTICO:

1. Falha em ao menos um módulo vital (suporte de vida, energia ou habitação)
2. Reserva de energia abaixo de 20% — limiar de risco imediato
3. Radiação acima de 13,0 mSv — limiar de perigo biológico

- **Expressão Booleana para Estado de ALERTA**

```

1 is_alerta = (not is_critico) and (sum(modulos.values()) < len(modulos) or sensores['energiaRestante'] < 40)

```

O estado de ALERTA exige que o sistema NÃO seja crítico e que haja ao menos um módulo não operacional (mesmo que não vital) ou que a energia esteja abaixo de 40% — faixa de atenção preventiva. Caso nenhuma condição seja verdadeira, o status retorna NORMAL.

- **Tabela de Verdade Resumida**

Condição	Falha Vital	Energia < 20%	Radiação > 13	Resultado
Crítico (ex. 1)	SIM	—	—	CRÍTICO
Crítico (ex. 2)	NÃO	SIM	—	CRÍTICO
Alerta	NÃO	NÃO	NÃO	ALERTA (se mod. falho ou energia < 40%)
Normal	NÃO	NÃO	NÃO	NORMAL

5. Alertas e Recomendações Automáticas

A função `gerenciarAlertas(status, situacao, previsao)` é responsável por três atividades encadeadas: geração de alertas classificados, registro na pilha de eventos críticos e produção de recomendações contextuais.

- **Geração e Priorização de Alertas**

Os alertas são criados com prioridade numérica (3 = crítico, 2 = alerta) e ordenados antes de ser inseridos na fila global:

1. Prioridade 3: falha de módulo detectada e energia < 20% ou radiação > 13;
2. Prioridade 2: energia entre 20% e 40% e alerta preditivo de tendência de queda.

```

1
2 # Ordenar alertas por prioridade (3=Crítico, 2=Alerta, 1=Normal)
3 alertas_ciclo.sort(key=lambda x: x['prioridade'], reverse=True)
4
5 # Adicionar à fila global
6 for a in alertas_ciclo:
7     fila_alertas_sistema.append(a['msg'])

```

- Registro na Pilha de Eventos Críticos

Sempre que um módulo é identificado como inoperante, o evento é registrado com timestamp na `pilha_eventos_criticos`. Isso permite que o operador acesse o último incidente de forma imediata, sem percorrer logs históricos.

- Recomendações

Status	Ações Recomendadas
CRÍTICO	Ação 1: Manter suporte à vida e comunicação de emergência. Ação 2: Desligar laboratório e sistemas n.
ALERTA	Ação 1: Redirecionar energia para habitat e carregamento. Ação 2: Verificar integridade dos módulos e r
NORMAL	Nenhuma ação imediata necessária.

6. Técnica de Previsão: Média Móvel com Ajuste de Tendência

- Algoritmo

```

1 def realizarPrevisao(dados_energia, janela=3):
2     """
3     Objetivo: Aplicar média móvel para prever a tendência de energia.
4     """
5     if len(dados_energia) < 2: return dados_energia[-1] if dados_energia else 0
6     ultimos = dados_energia[-janela:]
7     media = sum(ultimos) / len(ultimos)
8     tendencia = ultimos[-1] - ultimos[-2]
9     return round(max(0.0, min(100.0, media + tendencia)), 2)

```

O modelo combina duas informações:

1. Média dos últimos 3 registros — suaviza ruído pontual de leituras instáveis;

2. Tendência (delta entre os dois últimos pontos) — captura a direção de mudança recente.

O resultado é limitado ao intervalo [0, 100] para respeitar os limites físicos da variável.

- **Utilização no Pipeline**

A previsão é calculada antes do diagnóstico e exibida ao lado da leitura atual:

1. Na interface: "Energia Restante: 18% (Previsto: 17,33%)";
2. Na lógica de alertas: se o valor previsto for menor que o atual, um alerta preditivo de "tendência de queda" é enfileirado.

7. Simulador de Dados de Telemetria

O módulo `data/simuladorDados.py` gera o arquivo `dados.csv` com registros sintéticos para testes e demonstração do sistema.

- **Parâmetros Gerados**

Variável	Faixa	Distribuição
energiaRest(%)	0 – 100	Uniforme (inteiro)
energiaGerada(Kw/h)	1000 – 5200	Uniforme (float)
tempExterna(°C)	-50 – +50	Uniforme (float)
tempInterna(°C)	15 – 30	Uniforme (float)
nivelRadiacao(mSv)	0 – 20	Uniforme (float)
elVento(m/s)	0 – 20	Uniforme (float)
modulos (6x)	0 ou 1	Ponderada: 80% operacional, 20% falha
qldComunicacao	Estavel / Instavel	Ponderada: 80% / 20%

O avanço temporal é simulado com incrementos aleatórios de dias, horas e minutos, gerando séries realistas sem periodicidade fixa. A função `statusLog()` analisa os dados e registra automaticamente os eventos correspondentes na coluna `logEventos`.

8. Decisões Técnicas

- **Uso de deque em vez de list para a fila de alertas**

Uma lista Python poderia ser usada com `pop(0)` para simular FIFO, mas essa operação é $O(n)$ — cada remoção desloca todos os elementos. A deque garante $O(1)$ em ambas as extremidades, o que é essencial em sistemas de tempo real onde a fila pode crescer rapidamente em cenários críticos.

- **Conversão de tipos na ingestão do CSV**

A função `lerDados()` tenta converter cada valor para float ou int na leitura, mantendo str apenas para valores não numéricos. Isso elimina a necessidade de casting nas funções de diagnóstico e previsão, reduzindo erros de tipo em runtime.

- **Cores ANSI para saída no terminal**

A escolha de uma interface colorida no terminal — em vez de logs em texto puro — é uma decisão consciente de usabilidade operacional. Em ambientes de controle de missão, a distinção visual imediata entre estados crítico (vermelho), alerta (amarelo) e normal (verde) reduz o tempo de interpretação do operador em situações de pressão.

- **Índice -1 para acesso ao registro mais recente**

O sistema processa sempre o último registro do CSV (`idx = -1`). Esta decisão reflete o modelo de uso: o operador executa o sistema periodicamente e deseja ver o estado atual, não um replay histórico. A previsão, por sua vez, utiliza toda a série histórica disponível para calcular a tendência.

9. Conclusão

O sistema demonstra como a escolha criteriosa de estruturas de dados e a definição precisa de regras lógicas permitem construir sistemas de diagnóstico robustos e eficientes, mesmo sem dependências externas além da biblioteca padrão do Python.

Os principais aprendizados técnicos consolidados no projeto:

- A diferença prática entre list, deque e dict para padrões de acesso distintos
- A modelagem de estados complexos com álgebra booleana clara e testável
- A integração de previsão analítica simples para adicionar dimensão preditiva ao sistema
- O design de interfaces de terminal informativas e orientadas ao operador

O sistema está estruturado para crescimento: a adição de novos sensores, módulos ou algoritmos de previsão pode ser feita de forma incremental sem redesenho da arquitetura central.